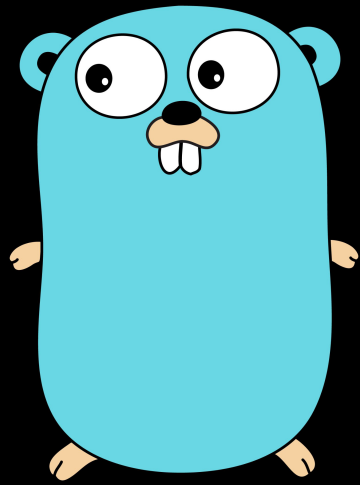
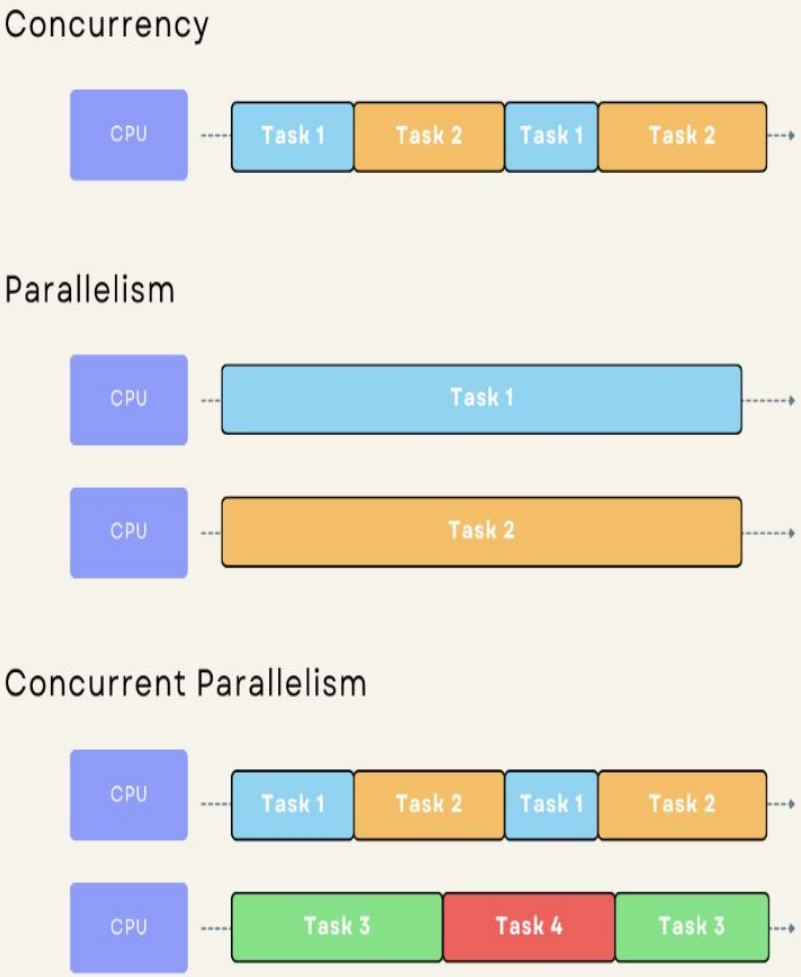
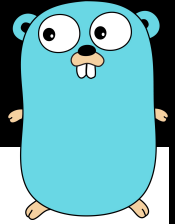
Go Concurrency

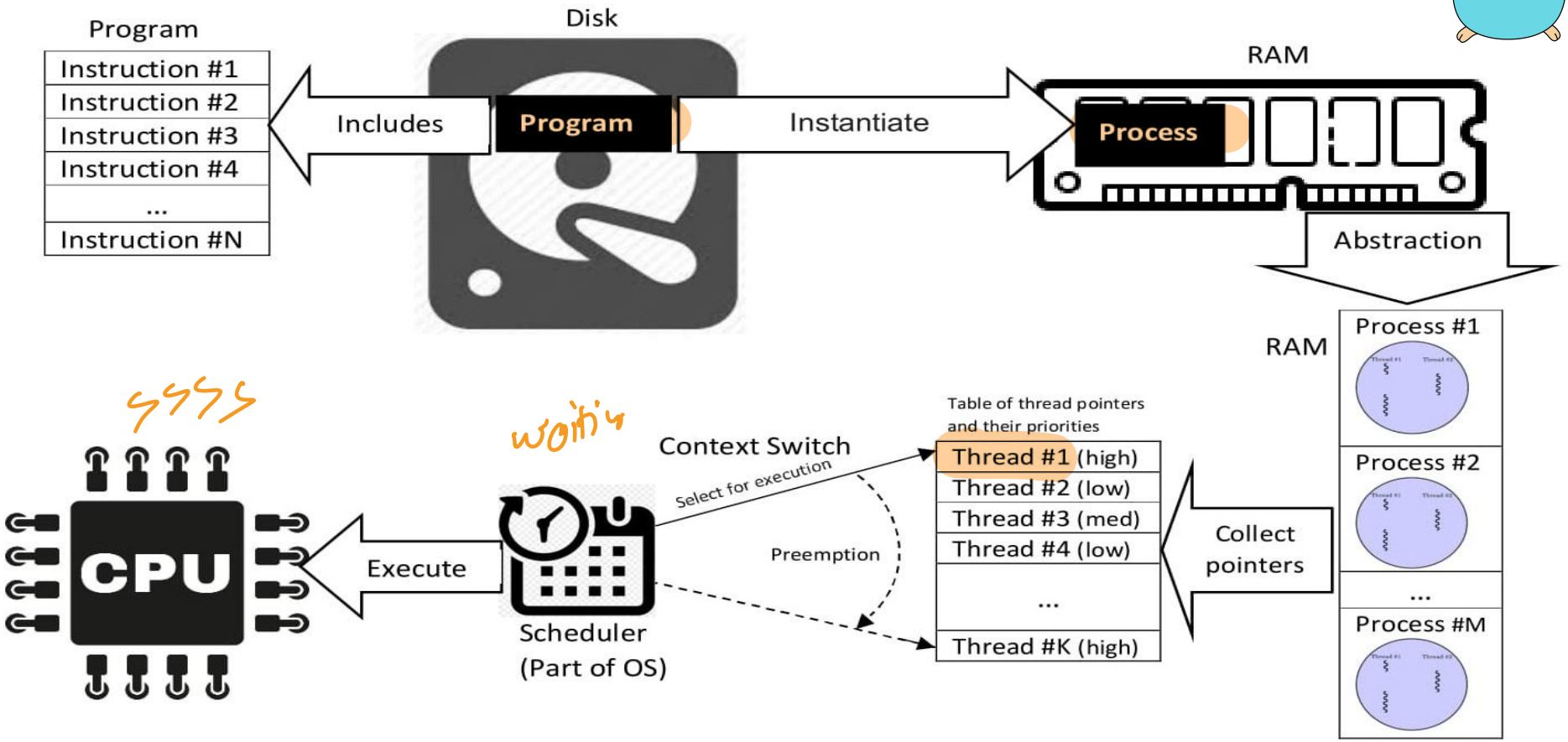


Intro to Concurrency



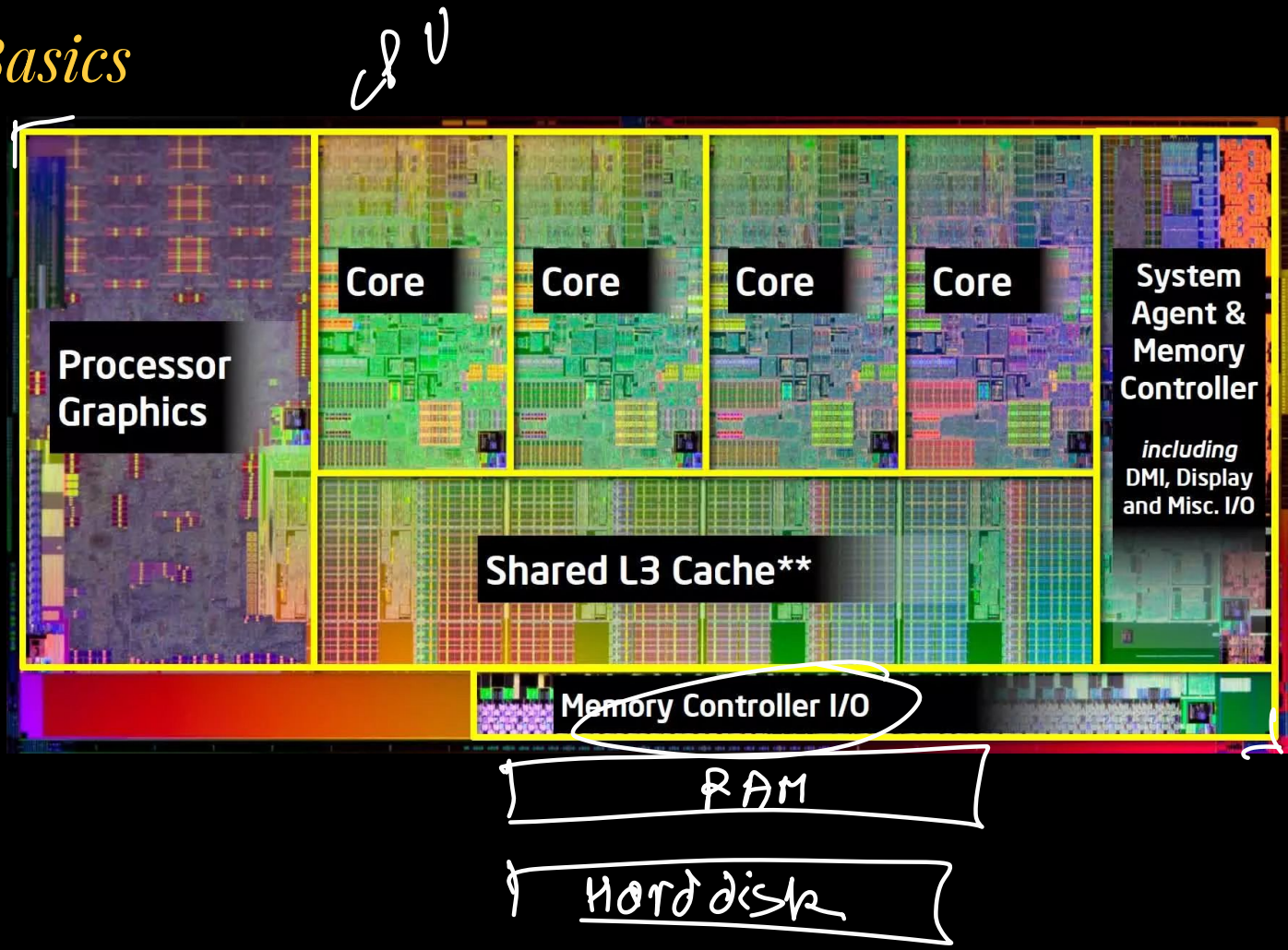
Concurrency: Basics

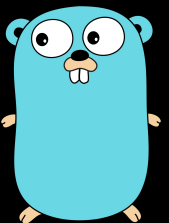




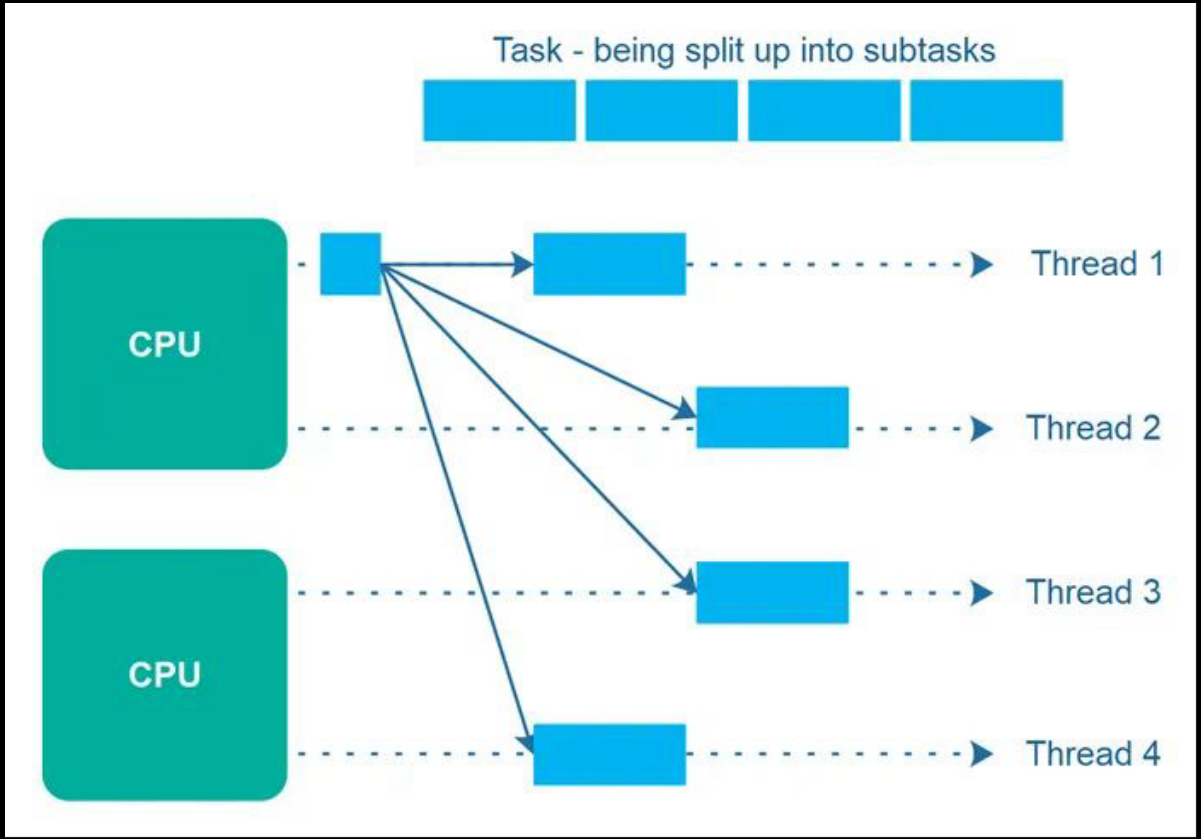
Concurrency: Basics

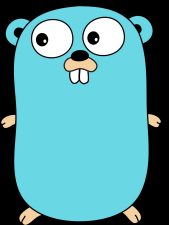
- CPU cores
- OS Threads
- Goroutines





Process vs Thread





Process vs Threads

Process: A process is an independent program in execution with its own memory space.

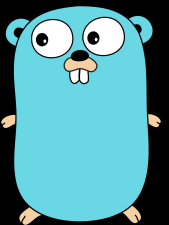
- Example: Chrome browser is a process

Thread: A thread is the smallest unit of execution inside a process.

- Chrome tabs are threads

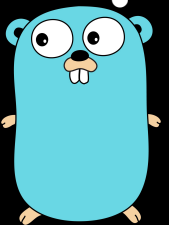
Important

- each process has separate memory space
- a process can have multiple threads sharing the same memory
- threads share the same process memory



Interview Questions: Process vs Threads

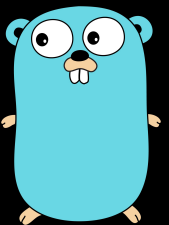
- Can threads exist without process?
  - *No, threads exist inside process.*
- Node.js runs as process or thread?
  - *Node.js runs as a process*
- Is each HTTP request a process?
  - *No, usually handled by thread or goroutine.*
- Why threads are called lightweight?
  - *Because they use less memory and share resources.*
- Why threads are preferred over process in web servers?
  - *Threads are preferred because they are lightweight, faster, and share memory, making request handling efficient.*



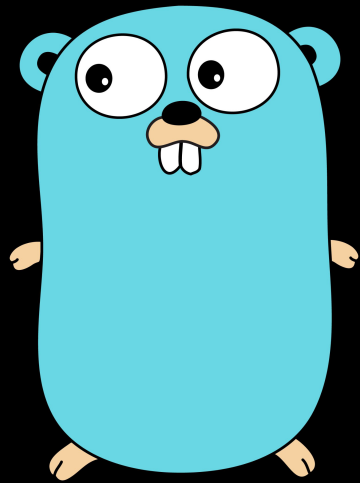
Interview Questions: Process vs Threads

Does Go create one thread per request?

- No. Go creates one goroutine per request. Many goroutines are multiplexed onto fewer OS threads.



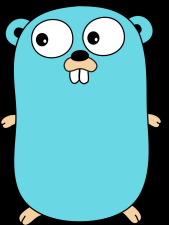
Go Routine

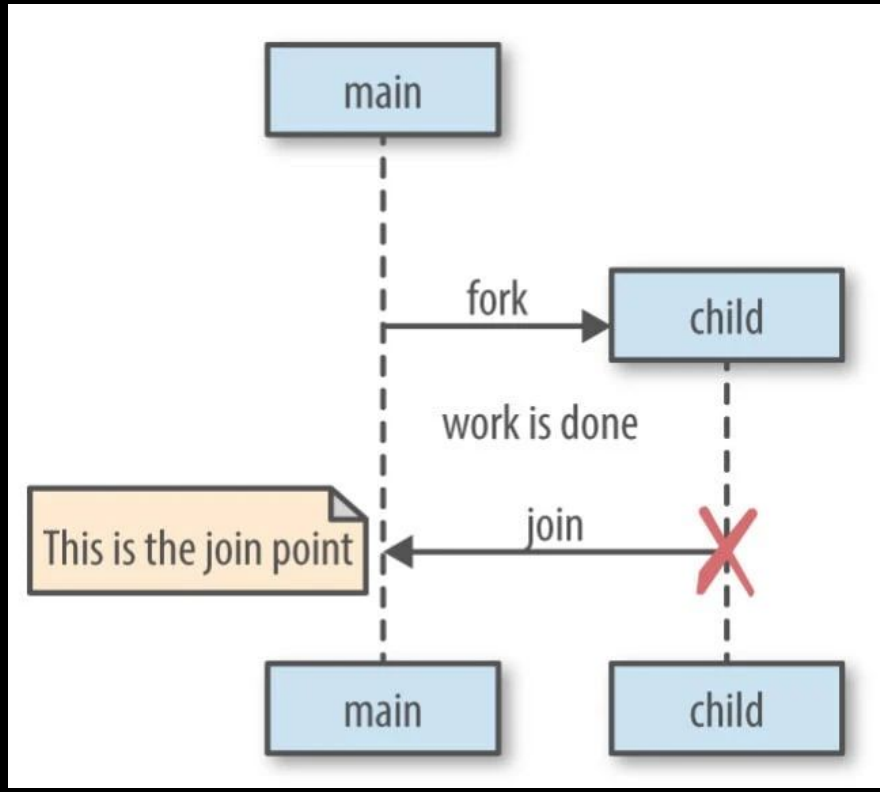


Goroutine

- A goroutine is a lightweight concurrent **function** managed by the Go runtime.
- Runs independently
- Much cheaper than threads
- Thousands or millions can exist
- **main function runs as the main goroutine**, all other goroutines are spawned from it.

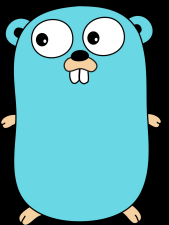
Syntax: `go functionName()`





Goroutine

- **Non Deterministic** execution
- Go statement does NOT guarantee execution order of goroutines. **Scheduler** decides based on:
  - *CPU availability*
  - *Timing*
  - *OS scheduling*
  - *Internal runtime state*

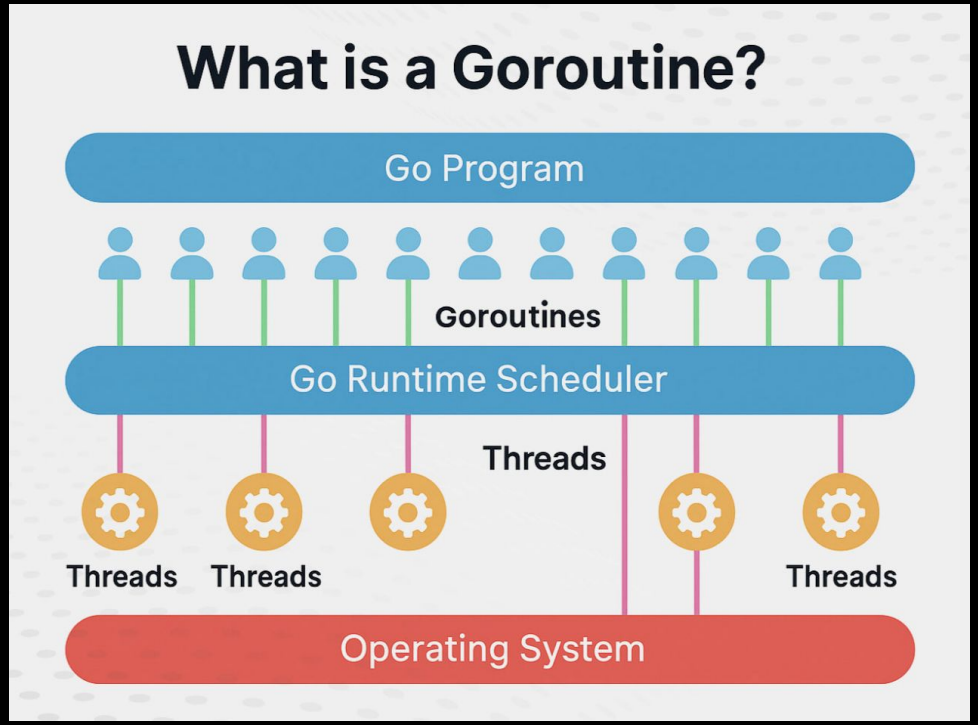


GMP model

GMP Scheduler model

- G: Goroutine
- M: Machine (OS Thread)
- P: Process (Logical CPU)

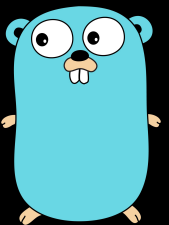
Go's GMP scheduler maps many **Goroutines (G)** onto a smaller number of **OS Threads (M)**, which run on **logical processors (P)** to execute concurrently.



Channel

- Channels are a typed conduit through which you can **send and receive values** with the channel operator **<-**
- Close channel **close(ch)**

```
ch <- v     // Send v to channel ch.
v := <-ch   // Receive from ch, and
            // assign value to v.
```

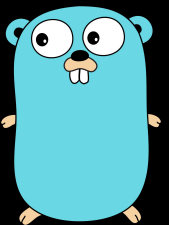


Select

- The select statement **lets a goroutine wait** on multiple communication operations.
- A select **blocks until one of its cases can run**, then it executes that case.
- It **chooses one at random** if multiple are ready.

```go
select {
case v1 := <-ch1:
    fmt.Println("From ch1:", v1)

case v2 := <-ch2:
    fmt.Println("From ch2:", v2)
}
```

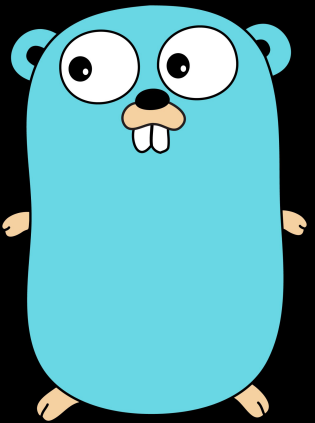


Interview Questions

- What is select in Go?
  - *A control structure that waits on multiple channels*
- Can select be used without channels?
  - *No. It only works with channel send/receive operations.*
- If multiple cases are ready, which one runs?
  - *One is chosen randomly.*

Checklist for Mid-sem

- **Array**
- **Slices**
- **Maps**
- **Struct**
- **Interface**

All the best

for mid-sem